Scaling Matrix Multiplication: From CUDA Kernels to Multi-GPU Tensor Parallelism

Aryan Jain* Fanyi Pu* Ze Hong Maxwell Au*
College of Computing and Data Science, Nanyang Technological University
*Equal contribution. Authors listed in alphabetical order.

SC4064 GPU Programming — Course Project Report

Abstract. General Matrix Multiplication (GEMM) is the computational backbone of modern deep learning. This report presents a systematic study across three scales: (i) progressive single-GPU CUDA kernel optimization through seven stages—from naive global memory access to warp-level tiling—benchmarked against cuBLAS on NVIDIA H100 GPUs; (ii) intra-node tensor parallelism on 8×H100 connected via NVLink; and (iii) cross-node tensor parallelism on 16×H100 spanning two nodes over 400 Gb/s InfiniBand. Our best custom kernel reaches 32.8 TFLOPS, 63% of cuBLAS throughput. Strong scaling on 8×H100 for $N = 16384$ yields a 7.03 × speedup (88% efficiency) at 360 TFLOPS aggregate; scaling to 16 GPUs across the IB fabric pushes the $N = 32768$ workload to 667 TFLOPS aggregate. As local GEMM kernels approach peak efficiency, the communication-to-computation ratio rises from 0.22 (naive) to 0.88 (cuBLAS), quantifying the crossover at which inter-GPU communication becomes the dominant bottleneck. A transport sweep confirms that NCCL's IB path is $\sim 125\times$ faster than the TCP fallback at the largest matrix size.

1 Introduction

GEMM operations underpin virtually all compute-intensive workloads in deep learning [1], [2]. In Transformer architectures, multi-head attention and feed-forward layers are fundamentally matrix multiplications. As models scale beyond the memory capacity of a single accelerator, *tensor parallelism* [3] has become indispensable, distributing weight matrices across GPUs at the cost of inter-GPU communication.

While vendor-tuned libraries such as cuBLAS [4] deliver near-optimal single-GPU performance, they abstract away the complex interaction between hardware-level compute intensity and system-level communication latency [5]. This project bridges that gap by:

1. Implementing seven progressively optimized CUDA GEMM kernels (plus one deliberately uncoalesced negative control) to understand hardware-level constraints (memory coalescing, shared memory tiling, register blocking, warp scheduling).
2. Building a distributed tensor-parallel linear layer (forward and backward) using NCCL, including a complete parallel MLP block and a communication–computation overlap variant.
3. Quantifying how local kernel efficiency impacts multi-GPU scalability, identifying the *crossover point* where communication dominates, and extending the study across the node boundary to measure how that crossover shifts when NVLink gives way to InfiniBand.

All experiments are conducted on the hardware described in Section 1.1.

1.1 Experimental Setup

Single-node experiments run on one node with 8 NVIDIA H100 80 GB SXM5 GPUs interconnected via NVLink 4.0 (NV18 topology, 900 GB/s bidirectional per GPU). Multi-node experiments span two such nodes (16 GPUs total) connected by 4×Mellanox ConnectX-7 400 Gb/s InfiniBand HCAs per node with GPUDirect RDMA. Both configurations use CUDA 13.1 and NCCL 2.29.3. The full hardware and software configuration is summarised in Table 1.

1.2 Benchmarking Methodology

Timing. Every measurement uses 5 discarded warmup iterations followed by 20 timed iterations. Each iteration is wall-clocked on the host with `std::chrono::high_resolution_clock` bracketing a `cudaStreamSynchronize` (or, in multi-GPU runs, a `std::thread::join` over all per-GPU worker threads), so the reported number is the wall-clock time of the slowest GPU. We report the arithmetic mean; error bars and standard-deviation columns are computed but (as noted in the respective captions) are omitted from plots whose variance is dominated by rare NCCL or CUDA-pool warm-up outliers rather than by genuine timing noise.

Decomposed timing. For every multi-GPU experiment we run three **independent** timed loops over the same buffers: a **GEMM-only** loop that calls the local kernel and syncs; a **Comm-only** loop that issues `ncclAllGather` (or `AllReduce`) and syncs; and a **Total** loop that runs the full `column_parallel_forward` (or equivalent). Reporting the first two separately makes the comm/compute ratio a direct, additive decomposition; the Total measurement captures any overlap or scheduling overhead that the two-loop sum misses.

Correctness. Every kernel is verified against a single-threaded CPU reference at $M = N = K = 256$ using matching pseudo-random inputs (xorshift with fixed seeds 42 and 137, scaled to $[-1, 1]$). A kernel passes if the maximum absolute element-wise error is below 10^{-5}; all nine kernels pass.

Multi-node launch. Cross-node runs are launched by the cluster scheduler as one process per node with rank 0 on the master and rank 1 on the worker; each process spawns 8 host threads, one per local GPU, and participates in a 16-way NCCL communicator initialised via `ncclCommInitRank`. Between experiments we issue a `cross_node_barrier` (an NCCL `AllReduce` on a dummy scalar) so that no measurement is contaminated by a neighbour's residual traffic.

Component	Specification
GPU (per node)	8 × NVIDIA H100 80 GB HBM3 (SXM5)
Intra-node interconnect	NVLink 4.0, all-to-all NV18 (900 GB/s bidirectional per GPU)
Inter-node interconnect	4 × Mellanox ConnectX-7 400 Gb/s InfiniBand per node, GPUDirect RDMA
GPU Compute	132 SMs, 1980 MHz boost, sm_90, 67 TFLOPS FP32 dense peak (FMA-counted)
GPU Memory	80 GB HBM3, ∼3.35 TB/s bandwidth per GPU
CPU (per node)	2 × Intel Xeon Gold 6448Y (32 cores / 64 threads each, 128 threads total)
System Memory (per node)	2 TB DDR5
OS	Ubuntu 24.04.4 LTS (kernel 5.14.0)
CUDA Toolkit	13.1
NCCL	2.29.3
GPU Driver	550.90.07
Max world size measured	16 GPUs (2 nodes × 8 GPUs)

Table 1: Hardware and software configuration. Single-node runs use one node with 8 GPUs over NVLink; multi-node runs span two nodes over 400 Gb/s InfiniBand.

Tests with `NCCL_DEBUG=INFO` confirm that the `auto/ib/ring` transports use `NET/IB` with GPUDirect RDMA over all four HCAs (`mlx5_{0,1,4,5}`), and that `tcp` falls back to the socket plugin.

2 System Design

The codebase has three loosely coupled layers: a **kernel layer** in which every GEMM implementation inherits a common `GemmKernel` interface and self-registers into a singleton `KernelRegistry` via a static initialiser (adding a new kernel is a single `.cu` file); a **CUDA resource layer** of move-only RAII wrappers (`CudaMemory<T>`, `DeviceMatrix`, `CudaStream`, `CudaEvent`, `CublasHandle`) that eliminate manual `cudaFree`/`cudaStreamDestroy` calls; and a **tensor-parallelism layer** that consumes kernels through the abstract interface and factors each parallel GEMM's transpose-and-multiply variants into two composable `grad_gemm_*` helpers. The benchmark binaries (`bench_single_gpu`, `bench_multi_gpu`, `bench_multi_node`) sit on top and share the same kernel registry and resource primitives.

3 Single-GPU Kernel Optimization

3.1 Optimisation Roadmap

We implement seven kernels, each introducing one major optimisation on top of the previous, plus an uncoalesced negative control. Every kernel is verified against a CPU reference at $M = N = K = 256$ (max abs. error $< 10^{-5}$). Table 2 summarises the hierarchy; the key observation is that each step either moves data accesses to a faster level of the memory hierarchy or amortises an existing load across more FLOPs.

3.2 Results and Analysis

Memory-bound tier. Naive and Coalesced plateau at ∼ 6.3 TFLOPS regardless of size: both are limited by the K global loads per FMA. The uncoalesced control at 498 GFLOPS ($N = 4096$) is 12.7 × slower than the coalesced variant, making the cost of bad access patterns concrete—a single axis swap in the thread index suffices to squander an order of magnitude of bandwidth. Shared-memory tiling lifts throughput to ∼ 9 TFLOPS by cutting HBM traffic by 32 ×, but smem bandwidth itself becomes the new ceiling.

Kernel	Key idea	Outcome
Naive	1 thread → 1 output, K global loads/FMA	$\mathcal{O}(1)$ FLOP/B
Coalesced	`threadIdx.x` → column (stride-1)	Fewer HBM transactions
Shared Mem	32 × 32 smem tiles reused 32 ×	32 × less HBM traffic
1D BlockTile	Thread coarsening, TM=8 rows per thread	Better smem reuse
2D BlockTile	Register blocking, 8 × 8 per thread	8 × fewer smem reads
Vectorized	`float4` loads + transposed smem	Fewer instructions
WarpTile	Block → warp → thread hierarchy	Better L1 locality
cuBLAS	Vendor reference (TF32 tensor-core path)	Upper bound

Table 2: Kernel optimisation roadmap. Each row adds one technique on top of the previous; Uncoalesced (not shown) swaps `threadIdx.x` onto the row as a negative control.

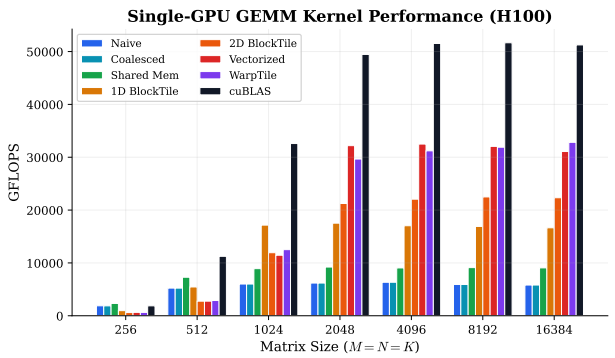

Figure 1: GFLOPS across all kernel stages and matrix sizes (Uncoalesced omitted for scale). Register-blocked kernels climb sharply with size; cuBLAS sits on a separate tier via its tensor-core path.

Compute-bound tier. Register blocking is the biggest single win: the 2D block tile reaches 22 TFLOPS at $N = 4096$ because each thread now performs 64 FMAs per smem pair, amortising the smem load cost. Vectorised `float4` loads push this to 32.5 TFLOPS, and the warp-tiled variant sustains 32.8 TFLOPS through $N = 16384$. cuBLAS peaks at 51.7 TFLOPS (77% of the 67 TFLOPS FP32 ceiling), 1.59 × our best hand-written kernel; the gap is attributable to TF32 tensor cores (which our kernels do not use) and offline autotuning of block shapes per (M, N, K).

Against H100's 67 TFLOPS FP32 ceiling (FMA-counted), cuBLAS achieves 77% of peak, the warp-tiled kernel 49%, and the naive kernel 9%. At SGEMM's operational intensity ($N/6 \approx 683$ FLOP/byte for $N = 4096$), the workload is deeply compute-bound—the ridge point sits at only 20 FLOP/byte—so the spread across kernels measures how well each fills the compute pipeline, not how well it amortises memory traffic.

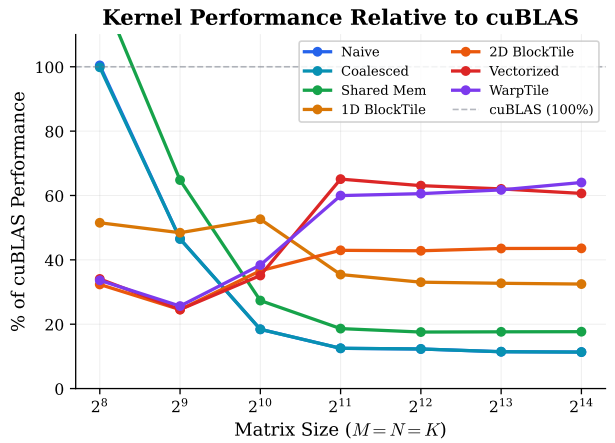

Figure 2: Each kernel as a percentage of cuBLAS. For $N \geq 2048$, the gap narrows monotonically from 11% (naive) to 63% (vectorised/warp). Small-size crossovers reflect library-dispatch overhead, not steady-state throughput.

4 Intra-Node Tensor Parallelism (NVLink)

Primitives. Consider a linear layer $Y = XW$ with $X \in \mathbb{R}^{M \times K}$, $W \in \mathbb{R}^{K \times N}$, sharded across p GPUs. In **column parallelism** we split $W = [W_1 \mid \cdots \mid W_p]$ along its output dimension, replicate X, and compute $Y_i = XW_i$ locally; the full output is assembled via `AllGather`. In **row parallelism** we split W along its input dimension and X accordingly, each GPU computes a partial product $Y_i = X_i W_i$, and the full output is an `AllReduce`-sum: $Y = \sum_i X_i W_i$. Column backward needs an `AllReduce` on $\mathrm{d}X$; row backward is entirely local. Following Megatron-LM [3] we compose column → row across an MLP block, which leaves exactly one `AllReduce` per forward and one per backward (Table 3) —the column output is already the correct input shard for the row follow-up, no inter-layer `AllGather` is needed.

We implement these primitives on top of NCCL [5]. Each GPU runs in a separate host thread with its own CUDA stream and cuBLAS handle; NCCL communicators for each target GPU count are pre-initialised via `ncclCommInitAll` at startup. Concretely the implementation includes:

- **Column-parallel forward/backward**: local GEMM + `ncclAllGather` (forward) / `ncclAllReduce` (backward on $\mathrm{d}X$).
- **Row-parallel forward/backward**: local GEMM + `ncclAllReduce` (forward) / no communication (backward).
- **Parallel MLP block**: composed column → row parallelism, matching the pattern in Table 3.
- **Communication–computation overlap**: the output matrix is chunked along the M dimension; each chunk's `AllReduce` is pipelined with the next chunk's GEMM on a separate CUDA stream, synchronised via `cudaEvent`.

All intra-node experiments use 8×H100 GPUs connected via NVLink 4.0. Backward passes that call our row-major custom kernels use `cublasSgeam` for explicit transposition, since those kernels lack a built-in transposed variant.

4.1 Scaling

Strong scaling. Figure 3 shows that $N = 16384$ scales from 171.6 ms (1 GPU) to 24.4 ms (8 GPUs), a 7.03 × speedup (**87.8% efficiency**, 360 TFLOPS aggregate). Past

Step	Operation	Per-GPU Compute	Communication
Fwd Layer 1	Column parallel	$H_i = XW_{1,i}$, $H_i \in \mathbb{R}^{M \times H/p}$	None
Fwd Activation	Element-wise	$H_i \leftarrow \sigma(H_i)$	None
Fwd Layer 2	Row parallel	$Y_i = H_i W_{2,i}$, $Y_i \in \mathbb{R}^{M \times N}$	`AllReduce`(Y)
Bwd Layer 2	Row backward	$\mathrm{d}W_{2,i}, \mathrm{d}H_i$ (local)	None
Bwd Layer 1	Column backward	$\mathrm{d}W_{1,i}$ (local), $\mathrm{d}X_i$ (partial)	`AllReduce`$(\mathrm{d}X)$

Table 3: Communication pattern in a parallel MLP block. Only two `AllReduce` operations are needed: one in the forward pass and one in the backward pass.

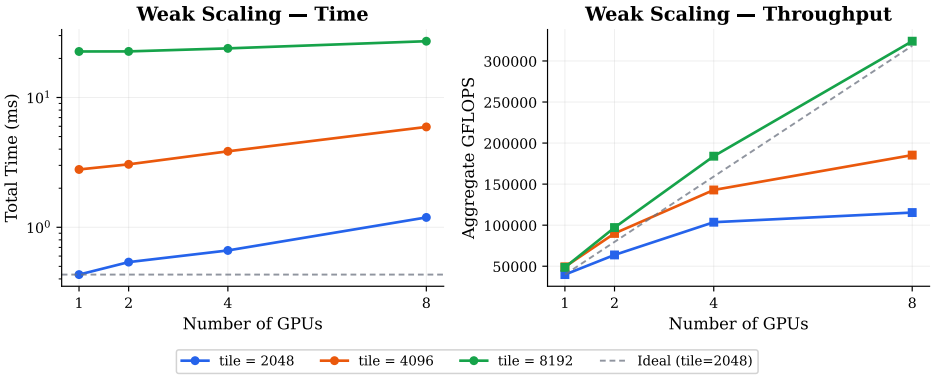

Figure 5: Weak scaling at three per-GPU tile sizes. Overhead is entirely from `AllGather`; throughput approaches linear as the tile grows.

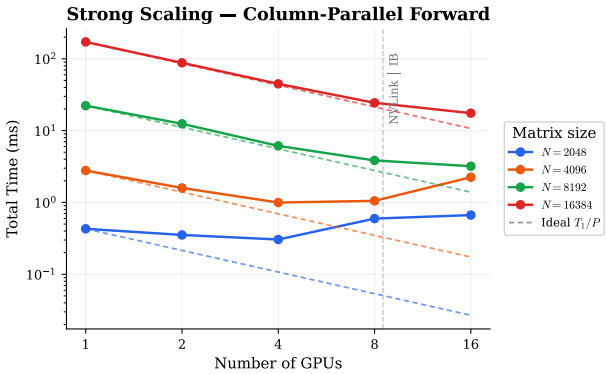

Figure 3: Strong scaling: total time vs. GPUs for four matrix sizes, combining NVLink (1–8 GPUs) and cross-node IB (16 GPUs). Dashed guides are ideal T_1/P.

4 GPUs the $N = 2048$ line inverts—the 16 MB `AllGather` takes longer than the shard-sized GEMM, so adding GPUs **increases** time. The jog at 8→16 GPUs for the small sizes marks the NVLink→IB transition.

Weak scaling. With per-GPU tile held fixed (Figure 5), the 2048 tile incurs a 2.75 × slowdown at 8 GPUs because `AllGather` latency dominates; the 8192 tile stays within 1.20 × of single-GPU time and reaches 324 TFLOPS aggregate.

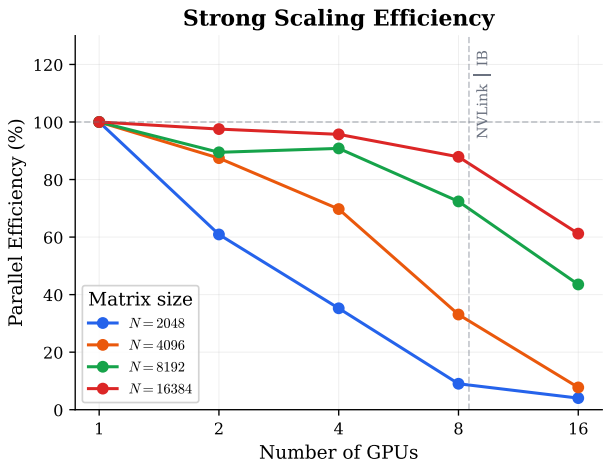

Figure 4: Parallel efficiency $\eta = T_1/(P \cdot T_P)$. $N = 16384$ holds 88% at 8 GPUs (NVLink) and 61% at 16 GPUs (cross-node IB); small matrices collapse within the NVLink domain.

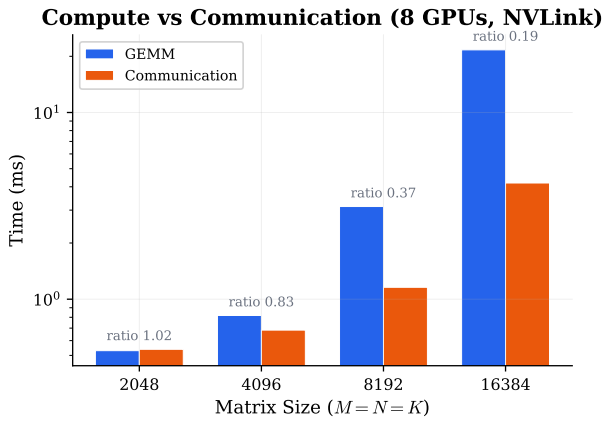

Figure 6: GEMM vs. communication time at different sizes on 8 GPUs (cuBLAS, NVLink). Ratio drops 1.02 → 0.19 as compute's $O(N^3)$ beats communication's $O(N^2)$.

4.2 Communication–Computation Analysis

The central question: *how does the comm-to-compute ratio evolve as local kernels are optimised, and as matrix size grows?*

Figure 6 captures the cubic-vs-quadratic law: at $N = 2048$ compute and comm are equal (0.53 vs 0.54 ms); by $N = 16384$ compute is 21.7 ms vs 4.2 ms comm, ratio 0.19. Modern Transformer hidden dimensions ($N \geq 8192$) therefore sit comfortably in the compute-dominated regime.

Key result. Figure 7: communication time is **constant** at ≈ 0.69 ms across kernels—it depends only on the `AllGather` payload (64 MB at $N = 4096$, 8 GPUs) and the NVLink fabric, not on the local GEMM. As the GEMM accelerates from 3.20 ms (Naive) to 0.78 ms (cuBLAS), the ratio climbs monotonically from 0.22 to 0.88. The interpretation is direct: with a naive kernel, further optimisation translates almost one-for-one into end-to-end speedup; with cuBLAS, even infinite GEMM speedup would buy at most 1.88 × end-to-end, because communication is already within a factor of two of compute. This crossover is the fundamental tension of distributed deep learning—local compute efficiency and interconnect efficiency must be co-optimised, and the "best" kernel depends on where the system currently sits on this curve.

4.3 MLP and Overlap

On H100 NVLink, `AllReduce` latency is only ~ 3 ms even for 1 GB, so the hideable window is narrow; the next section

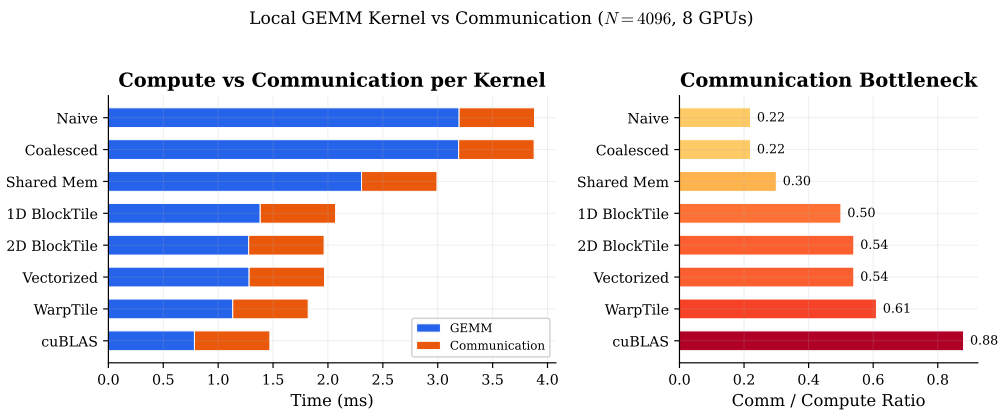

Figure 7: Left: stacked GEMM and comm time per kernel ($N = 4096$, 8 GPUs). Right: comm/compute ratio rising 0.22 (Naive) → 0.88 (cuBLAS) as the local GEMM accelerates.

shows that overlap becomes considerably more valuable when fabric latency rises.

5 Cross-Node Tensor Parallelism (InfiniBand)

We repeat the column-parallel strong-scaling and ratio-vs-size experiments on 16 GPUs spanning two nodes, sweeping NCCL across four transport configurations in a single run: `auto` (NCCL default), `ib` (`NCCL_NET=IB`), `ring` (`NCCL_ALGO=Ring`), and `tcp` (`NCCL_IB_DISABLE=1`, forcing the socket plugin). The IB paths use `NET/IB` with GPUDirect RDMA over all four HCAs; `tcp` falls back to plain sockets as intended.

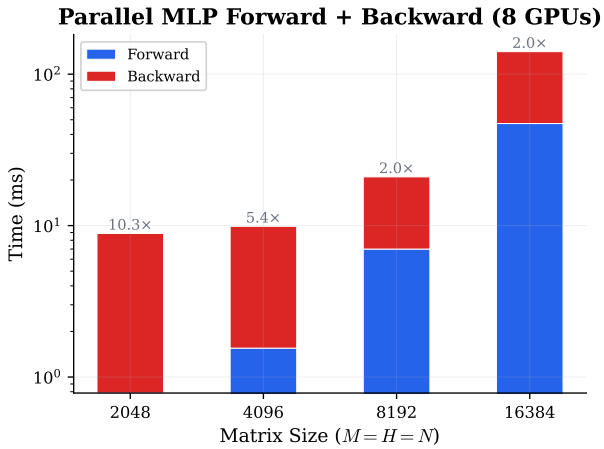

Figure 8: Parallel MLP (column → row) fwd + bwd on 8 GPUs. At $N \geq 8192$ the bwd/fwd ratio converges to its 2 × algorithmic limit (2 GEMMs fwd vs 4 GEMMs bwd); small sizes inflate it via per-call launch overhead.

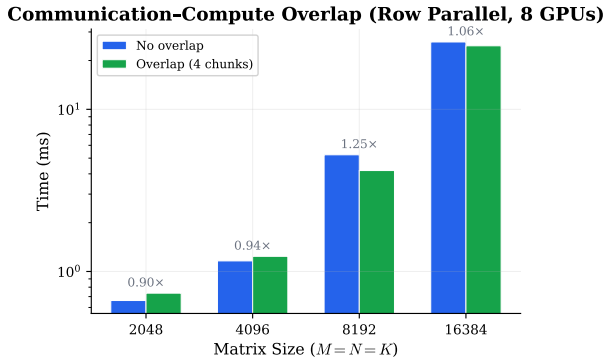

Figure 9: Row-parallel forward with and without 4-chunk overlap. Overlap hurts (0.90–0.94 ×) at small sizes (sync overhead dominates) and helps modestly at $N = 16384$ (1.06 ×, low variance). The $N = 8192$ point shows 1.25 × but has 73% CoV on the baseline and is unreliable.

Transport. Figure 10 is the headline multi-node result. At $N = 32768$, the IB transports move the 4 GB `AllGather` in 23–25 ms (∼ 175 GB/s effective goodput per GPU), whereas TCP takes 2.93 *seconds*—a 125 × slowdown that fully dominates every other cost. TCP's ratio stays above

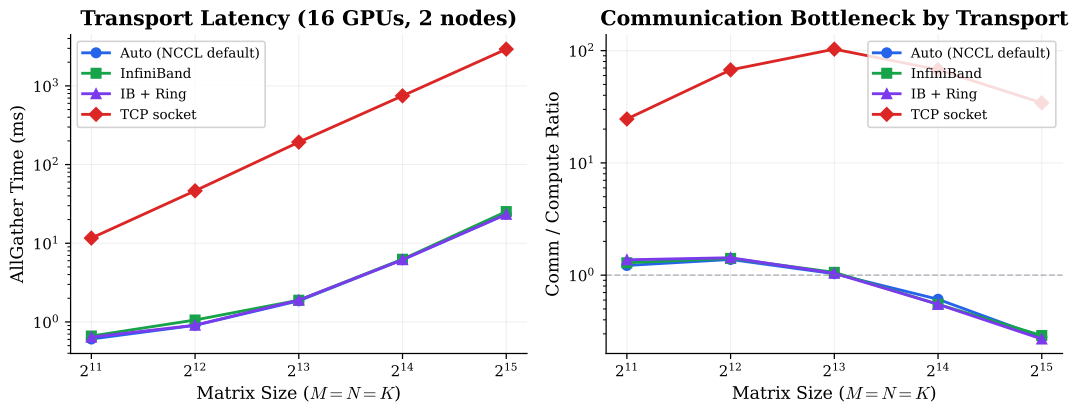

Figure 10: Left: per-size `AllGather` time at 16 GPUs across four transports. Right: resulting comm/compute ratio. IB, auto, and ring trace the same curve; TCP lies two orders of magnitude above.

30 across the entire range, meaning local kernel speed is invisible in that regime.

Scaling across the node boundary. The 16-GPU points in Figure 3 and Figure 4 come from the `auto` (IB) run. $N = 16384$ drops from 24.4 ms (8 GPUs, NVLink) to 17.5 ms (16 GPUs, IB)—a further 1.39 × speedup, efficiency 61%. $N = 32768$ on 16 GPUs reaches **667 TFLOPS** aggregate, the highest throughput we measure anywhere, with ratio 0.28 (still compute-dominated). Small matrices go the other way —$N \leq 4096$ is slower on 16 IB GPUs than on 8 NVLink GPUs—reinforcing the design rule: choose TP width so the sharded GEMM stays comfortably to the left of the bandwidth-limited `AllGather`. On this cluster, that threshold is $N \approx 8192$ for NVLink-only and $N \approx 16384$ once IB is on the critical path.

6 Discussion and Conclusion

Summary. Across three scales—single GPU, single-node NVLink, cross-node IB—the central result is that communication and computation trade off in a predictable way. Seven progressively optimised kernels trace the path from 6.3 TFLOPS (memory-bound) to 32.8 TFLOPS (63% of cuBLAS); 8×H100 strong scaling at $N = 16384$ delivers 7.03 × speedup (88% efficiency, 360 TFLOPS aggregate). As the local GEMM accelerates, the comm/compute ratio climbs from 0.22 (Naive) to 0.88 (cuBLAS)—a direct measurement of the crossover where the fabric starts to dominate. Adding a second node over 400 Gb/s IB pushes $N = 32768$ to 667 TFLOPS aggregate, but forcing NCCL onto TCP inflates `AllGather` by 125 × and drives the ratio past 100.

Limitations. Our kernels are FP32-only and do not use tensor cores; the 37% gap to cuBLAS reflects that rather than any algorithmic deficit. The weak-scaling sweep goes up to a per-GPU tile of 8192 only, above which the full matrix (on 8 GPUs) exceeds device memory. Overlap is measured with row-parallel `AllReduce` only; gains on column-parallel `AllGather` (which is less latency-hidden) may differ. Finally, the multi-node study covers two nodes with uncongested IB; a larger cluster, or one sharing the fabric with other jobs, might yield different `AllGather` characteristics.

Takeaway. The data support a simple design rule: choose tensor-parallel width so the sharded GEMM stays comfortably to the left of the bandwidth-limited collective—and verify the interconnect is actually doing what you think it is. On this cluster, that threshold is $N \approx 8192$ within a NVLink island and $N \approx 16384$ once IB is on the critical path. Below those thresholds, adding GPUs is counter-productive; above them, tensor parallelism yields a near-linear speedup even across the node boundary.

Bibliography

[1] A. Vaswani *et al.*, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[2] Z. Jia, M. Maggioni, B. Staiger, and D. P. Scarpazza, "Dissecting the NVIDIA volta GPU architecture via microbenchmarking," *arXiv preprint arXiv:1804.06826*, 2018.

[3] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

[4] NVIDIA Corporation, "cuBLAS Library." 2026. [Online]. Available: https://docs.nvidia.com/cuda/pdf/CUBLAS_Library.pdf

[5] NVIDIA Corporation, "NCCL Developer Guide." 2026. [Online]. Available: https://docs.nvidia.com/deeplearning/nccl/user-guide/